E5 — Mise en situation professionnelle

Certification Ingénieur en Infrastructure Massive de Données

Candidat : Mezine Elias

Bloc de compétences : Bloc 3 — Élaborer et maintenir un entrepôt de données

Livrable : E5 — Mise en situation

Sommaire

Modéliser la structure des données d'un entrepôt de données **3**
1. Données nécessaires aux analyses . . . 3
   1.1 Source des données . . . 3
   1.2 Liste exhaustive des données . . . 3
   1.3 Données calculées pour l'analyse . . . 3
2. Modélisation . . . 4
   2.1 Schéma en étoile vs alternatives . . . 4
   2.2 Méthode Kimball : Bottom-up . . . 4
   2.3 Modèle Conceptuel de Données (MCD) . . . 4
   2.4 Modèle Logique de Données (MLD) . . . 5
   2.5 Modèle Physique de Données (MPD) . . . 5

Créer un entrepôt de données **6**
1. Fonctionnalités de l'entrepôt de données . . . 6
2. Configurations principales . . . 7
   2.1 Infrastructure Snowflake . . . 7
   2.2 Orchestration . . . 7
   2.3 Paramétrage GitHub Actions . . . 7
   2.4 Transformation (dbt) . . . 8
3. Gestion des accès et des rôles . . . 8
   3.1 Rôles système Snowflake . . . 8
   3.2 Rôles pour les équipes analytiques . . . 9
4. Procédure de test . . . 9
   4.1 Qualité des données . . . 9
   4.2 Qualité du code . . . 9
5. Documentation technique . . . 12
   5.1 Documentation MkDocs . . . 12
   5.2 Documentation dbt . . . 12
   5.3 Structure et accessibilité . . . 13
6. Choix et retour d'expérience sur la pile technique . . . 13

Intégrer les ETL nécessaires en entrée et en sortie d'un entrepôt de données **14**
1. Ingestion Python 14
2. Transformations dbt 14
2.1 Staging (`yellow_taxi_trips_stg.sql`) 14
2.2 Modèle dimensionnel 14
2.3 Marts 14

Synthèse **15**

Modéliser la structure des données d'un entrepôt de données

1. Données nécessaires aux analyses

1.1 Source des données

Les données proviennent du **TLC Trip Record Data** de la Taxi & Limousine Commission de New York City, accessibles à l'adresse : https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Les fichiers sont au format **Parquet** et couvrent les trajets des taxis jaunes de NYC.

*La TLC ne garantit pas l'exactitude de ces données. Le projet est distribué sous licence **MIT**.*

1.2 Liste exhaustive des données

Données transactionnelles :

Colonne	Type	Description
TRIP_ID	NUMBER	Identifiant unique du trajet
VENDORID	NUMBER	Identifiant du fournisseur technologique
TPEP_PICKUP_DATETIME	TIMESTAMP_NTZ	Date et heure de prise en charge
TPEP_DROPOFF_DATETIME	TIMESTAMP_NTZ	Date et heure de dépose
PASSENGER_COUNT	NUMBER	Nombre de passagers
TRIP_DISTANCE	NUMBER	Distance du trajet en miles
RATECODEID	NUMBER	Code tarifaire appliqué
STORE_AND_FWD_FLAG	BOOLEAN	Indicateur store-and-forward
PULOCATIONID	NUMBER	Identifiant de zone de prise en charge
DOLOCATIONID	NUMBER	Identifiant de zone de dépose
PAYMENT_TYPE	NUMBER	Type de paiement
FARE_AMOUNT	NUMBER	Montant de base du trajet
EXTRA	NUMBER	Suppléments heures de pointe et nuit
MTA_TAX	NUMBER	Taxe MTA
TIP_AMOUNT	NUMBER	Montant du pourboire
TOLLS_AMOUNT	NUMBER	Montant des péages
IMPROVEMENT_SURCHARGE	NUMBER	Surcharge d'amélioration
TOTAL_AMOUNT	NUMBER	Montant total facturé au passager
CONGESTION_SURCHARGE	NUMBER	Surcharge de congestion
AIRPORT_FEE	NUMBER	Frais d'aéroport
CBD_CONGESTION_FEE	NUMBER	Frais de congestion CBD

1.3 Données calculées pour l'analyse

Des métriques intermédiaires sont calculées par le modèle `int_trip_metrics` depuis les données brutes :

Métrique	Calcul
trip_duration_minutes	Différence en minutes entre pickup et dropoff
pickup_hour	Heure extraite du pickup datetime
pickup_day	Jour extrait du pickup datetime
pickup_month	Mois extrait du pickup datetime
avg_speed_mph	(distance * 3600) / durée en secondes
tip_pct	tip_amount / fare_amount

2. Modélisation

2.1 Schéma en étoile vs alternatives

Le schéma en étoile a été retenu comme pattern de modélisation. Il a été préféré au schéma en flocon ou en constellation car :

- **Simplicité des jointures** : Les requêtes analytiques ne nécessitent qu'un seul niveau de jointure entre la table de faits et chaque dimension, ce qui optimise les performances sur Snowflake.
- **Dimensions plates** : Les dimensions temporelles — date et heure — sont des référentiels plats, sans hiérarchie complexe nécessitant un flocon.
- **Domaine unique** : Le projet couvre un seul domaine métier — les trajets de taxi — ce qui ne justifie pas une constellation.

2.2 Méthode Kimball : Bottom-up

La méthode **Kimball** structure l'entrepôt de données en data marts domaine par domaine, chacun organisé en schéma en étoile. L'approche **bottom-up** a été retenue pour les raisons suivantes :

- **Volume de données** : les données NYC Taxi représentent plusieurs centaines de millions de lignes depuis 2009. L'approche bottom-up permet de construire un premier data mart opérationnel rapidement, sans attendre la modélisation complète de l'ensemble du SI.
- **Fréquence de mise à jour** : le pipeline s'exécute le 1er de chaque mois. L'approche incrémentale permet de ne traiter que les nouvelles données à chaque exécution mensuelle.
- **Nature des analyses** : les besoins portent sur un domaine unique et bien défini — les trajets de taxis. Le schéma en étoile est optimal pour les requêtes OLAP temporelles.
- **Source unique** : le projet s'appuie sur une seule source — le site NYC TLC. L'approche bottom-up est particulièrement adaptée lorsqu'on part d'une source bien identifiée pour construire des data marts spécifiques.
- **Livraison rapide** : contrairement à l'approche top-down qui nécessite la modélisation préalable de l'ensemble du SI, l'approche bottom-up livre de la valeur analytique dès le premier data mart fonctionnel.

2.3 Modèle Conceptuel de Données (MCD)

Le MCD représente les entités et leurs associations de façon indépendante de toute technologie. Les trois modèles — MCD, MLD et MPD — ont été réalisés avec **Looping**. Looping étant disponible uniquement sous Windows, l'exécution sous Linux a nécessité **Wine**, un émulateur Windows.

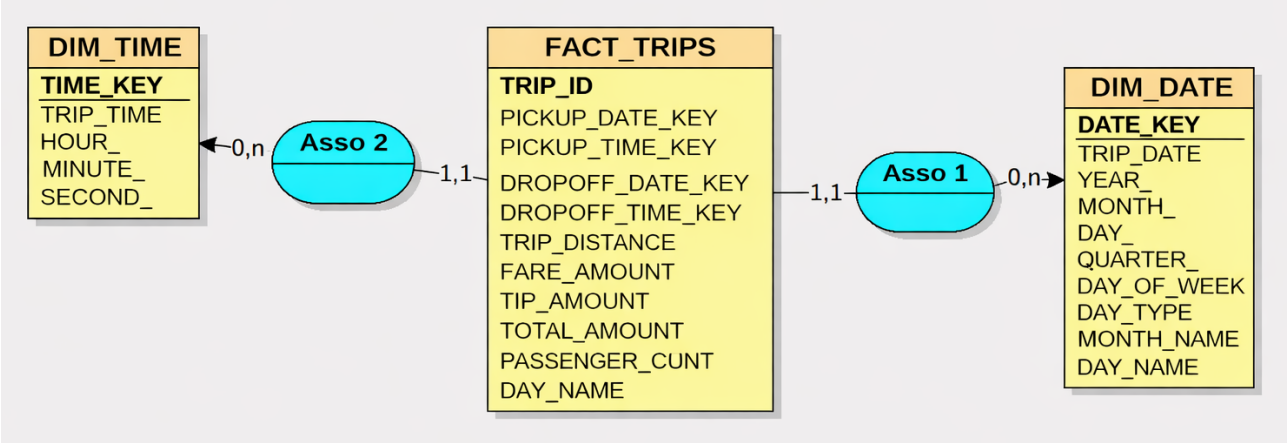

Figure 1: MCD du schéma en étoile dans Snowflake

2.4 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en tables relationnelles. Chaque relation entre la table de faits et une dimension est de type **1,1 — 0,N** : un trajet est associé à une et une seule date/heure, mais une date/heure peut être associée à zéro ou plusieurs trajets. Du côté du fait la cardinalité est toujours 1,1, ce qui fait disparaître les tables d'association du MCD : les clés primaires des dimensions migrent directement en clés étrangères dans la table de faits.

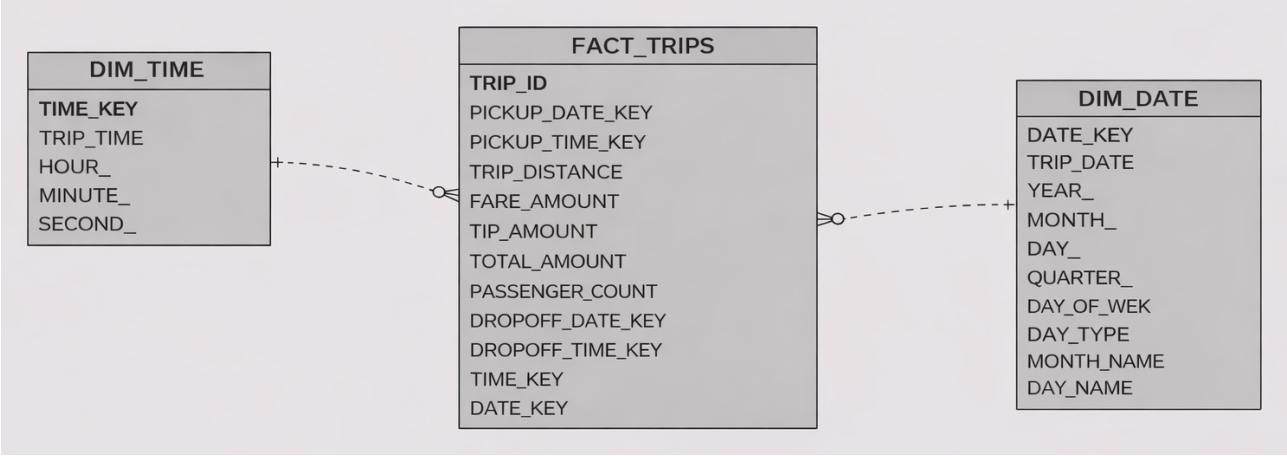

Figure 2: MLD du schéma en étoile dans Snowflake

Fait (FK)	Dimension	Clé (PK)	Cardinalité
pickup_date_key	dim_date	date_key	N:1
dropoff_date_key	dim_date	date_key	N:1
pickup_time_key	dim_time	time_key	N:1
dropoff_time_key	dim_time	time_key	N:1

2.5 Modèle Physique de Données (MPD)

Le MPD traduit le MLD en structures concrètes propres au SGBD cible. **Snowflake** détermine les types physiques utilisés — NUMBER, TIMESTAMP_NTZ, VARCHAR et BOOLEAN — ainsi que les spécificités de stockage : table permanente pour SCHEMA_RAW et SCHEMA_FINAL, table transitoire pour SCHEMA_STAGING, vues pour les marts et les modèles intermédiaires.

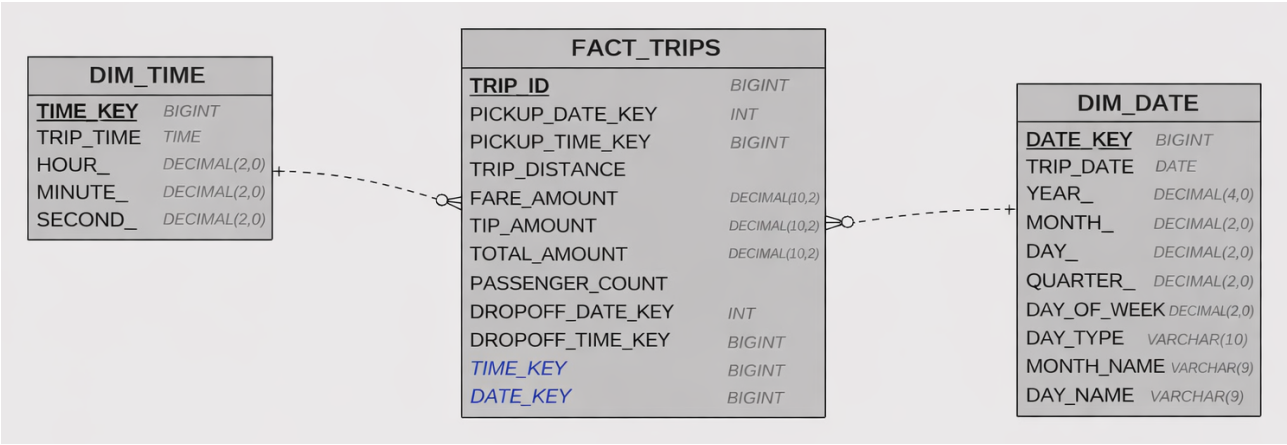

Figure 3: MPD du schéma en étoile dans Snowflake

Créer un entrepôt de données

1. Fonctionnalités de l'entrepôt de données

L'entrepôt de données **NYC_TAXI_DW** hébergé sur **Snowflake** remplit les fonctionnalités suivantes :

- **Ingestion automatisée** de fichiers Parquet depuis le site NYC TLC
- **Stockage structuré** en 3 couches : RAW, STAGING et FINAL
- **Transformation et nettoyage** des données via dbt
- **Modèle dimensionnel en étoile** pour les analyses OLAP
- **7 marts analytiques temporels** en sortie
- **Gestion des rôles et accès** pour le pipeline de transformation

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	TABLE_TYPE	IS_TRANSIENT	ROW_COUNT	CREATED	LAST_ALTERED	LAST_DDL	LAST_DDL_BY
1	NYC_TAXI_DW	SCHEMA_STAGING	TAXI_ZONE_LOOKUP	BASE TABLE	NO	265	2025-12-23 05:40:11.071 -080(	2025-12-23 05:40:13.447 -080(	2025-12-23 05:40:11.071 -080(	USER_DEV
2	NYC_TAXI_DW	SCHEMA_STAGING	INT_TRIP_METRICS	VIEW	null	null	2025-12-23 05:40:34.299 -080(	2025-12-23 05:40:34.420 -080(	2025-12-23 05:40:34.299 -080(	USER_DEV
3	NYC_TAXI_DW	SCHEMA_STAGING	YELLOW_TAXI_TRIPS_STG	BASE TABLE	YES	41110191	2025-12-23 05:40:20.630 -080(	2025-12-23 05:40:31.389 -080(	2025-12-23 05:40:20.630 -080(	USER_DEV

Figure 4: Schéma Staging dans Snowflake

NYC_TAXI_DW / SCHEMA_FINAL / **YEARLY_ANALYSIS**

View TRANSFORMER 8 minutes ago

View Details Columns Data Preview

COMPUTE_WH Updated just now

	YEAR	TRIP_COUNT	TOTAL_REVENUE	YOY_GROWTH
1	2007	2	45.50	null
2	2009	7	366.63	7.0578
3	2024	42	1178.34	2.2140
4	2025	64272056	1839663772.72	1561232.4069

Figure 5: Analyse annuelle (mart YEARLY_ANALYSIS)

NYC_TAXI_DW / SCHEMA_FINAL / **TIME_PERIOD_ANALYSIS**

View TRANSFORMER 8 minutes ago

View Details Columns Data Preview

COMPUTE_WH Updated just now

	DAY_OF_WEEK	IS_WEEKEND	TRIP_COUNT	TOTAL_REVENUE	AVG_REVENUE	AVG_DURATION	AVG_DISTANCE	AVG_SPEED
1	7	TRUE	9954166	267074618.18	26.83	16.08	6.80	24.14
2	2	FALSE	7504135	224743125.15	29.95	17.11	6.91	25.35
3	3	FALSE	8927635	255400470.99	28.61	17.28	6.20	21.11
4	4	FALSE	9814045	280853063.18	28.62	17.55	6.19	21.69
5	5	FALSE	10129142	294535282.28	29.08	18.04	5.96	20.17
6	6	FALSE	9708542	277854485.14	28.62	17.52	6.46	22.75
7	1	TRUE	8234442	239204318.27	29.05	15.94	6.92	25.61

Figure 6: Analyse par période (mart TIME_PERIOD_ANALYSIS)

NYC_TAXI_DW / SCHEMA_FINAL / TIME_OF_DAY_ANALYSIS

View TRANSFORMER 9 minutes ago

View Details Columns Data Preview

COMPUTE_WH Updated just now

	TIME_OF_DAY	TRIP_COUNT	AVG_REVENUE	AVG_DURATION	AVG_TIP
1	Evening	22577761	28.34	15.76	3.02
2	Night	5600587	28.25	14.45	2.39
3	Afternoon	22372825	29.78	19.07	3.35
4	Morning	13720934	27.37	17.18	2.86

Figure 7: Analyse par moment de la journée (mart TIME_OF_DAY_ANALYSIS)

2. Configurations principales

2.1 Infrastructure Snowflake

Le script d'initialisation `init_infra_snowflake.py` exécute séquentiellement : 1. Création du warehouse, database et schémas 2. Création des rôles et utilisateurs 3. Attribution des privilèges 4. Configuration du Time Travel

2.2 Orchestration

Le pipeline est orchestré par **GitHub Actions**, déclenché automatiquement le 1er de chaque mois à 10h UTC ou manuellement via `workflow_dispatch`. Chaque exécution s'effectue dans un environnement Ubuntu latest avec Python 3.10. Le projet est entièrement reproductible : forker le dépôt depuis GitHub et configurer les secrets suffit à lancer le pipeline immédiatement, sans infrastructure supplémentaire.

2.3 Paramétrage GitHub Actions

Le site NYC TLC est accessible publiquement sans authentification — aucun secret n'est requis pour la collecte des fichiers Parquet. Les secrets ci-dessous couvrent exclusivement l'accès à Snowflake.

Secrets obligatoires

Secret	Description
`SNOWFLAKE_ACCOUNT`	Identifiant du compte Snowflake
`SNOWFLAKE_USER`	Nom d'utilisateur Snowflake
`SNOWFLAKE_PASSWORD`	Mot de passe utilisateur Snowflake
`PASSWORD_DEV`	Mot de passe de l'utilisateur de développement
`PASSWORD_BI`	Mot de passe utilisateur Analyste BI
`PASSWORD_DS`	Mot de passe utilisateur Data Scientist
`PASSWORD_MC`	Mot de passe utilisateur Consommateur Mart

Variables configurables

Variable	Valeur par défaut	Description
`SCRAPING_YEAR`	année courante	Année de début du scraping
`WH_NAME`	`NYC_WH`	Nom du data warehouse
`DW_NAME`	`NYC_TAXI_DW`	Nom de la base de données
`RAW_SCHEMA`	`RAW`	Schéma des données brutes

Variable	Valeur par défaut	Description
`STAGING_SCHEMA`	`STAGING`	Schéma des données nettoyées
`FINAL_SCHEMA`	`FINAL`	Schéma des données finales
`COMPUTE_SIZE`	`X-SMALL`	Puissance de calcul de l'entrepôt de données
`RAW_TABLE`	`YELLOW_TAXI_TRIPS_RAW`	Table des données brutes
`STAGING_TABLE`	`YELLOW_TAXI_TRIPS_STG`	Table de staging
`TIMEZONE`	`UTC`	Fuseau horaire
`RETENTION_DAYS`	`1`	Durée de conservation Time Travel

`SCRAPING_YEAR` : par défaut l'année courante si le mois est supérieur ou égal à avril, l'année précédente sinon, afin de récupérer toutes les données disponibles depuis cette année jusqu'à aujourd'hui.

`COMPUTE_SIZE` : le warehouse se suspend automatiquement après 60s d'inactivité par défaut et reprend à la demande suivante. Snowflake facturant chaque utilisation au minimum 60s, cette valeur évite tout coût inutile entre les exécutions mensuelles.

`RETENTION_DAYS` : configuré à 1 jour par défaut pour limiter les coûts de stockage sur compte Standard, où Time Travel au-delà d'1 jour n'est pas disponible.

2.4 Transformation (dbt)

- **Version dbt** : dbt-snowflake 1.10.3
- **Packages dbt** : dbt_utils >=1.3.0, dbt_expectations >=0.10.0, dbt_date >=0.10.0

Nom de la table	Schéma	Type	Matérialisation
YELLOW_TAXI_TRIPS_RAW	SCHEMA_RAW	Permanente	Incrémentale
YELLOW_TAXI_TRIPS_STG	SCHEMA_STAGING	Transitoire	Incrémentale
int_trip_metrics	SCHEMA_STAGING	—	Vue
fact_trips	SCHEMA_FINAL	Permanente	Incrémentale
dim_date	SCHEMA_FINAL	Permanente	Table
dim_time	SCHEMA_FINAL	Permanente	Table
7 marts temporels	SCHEMA_FINAL	—	Vues

Les tables permanentes couvrent les données à conserver durablement : RAW comme source de vérité pour tout rejeu, FINAL pour les analyses continues. Les tables STAGING sont transitoires car reconstructibles depuis RAW, ce qui supprime le Fail-safe Snowflake et réduit le coût de stockage. La matérialisation incrémentale évite de retraiter plusieurs centaines de millions de lignes à chaque exécution mensuelle. Les vues n'ajoutent aucun coût de stockage et restent toujours à jour.

3. Gestion des accès et des rôles

L'entrepôt de données repose sur deux catégories de rôles. Les **rôles système Snowflake** sont prédéfinis par la plateforme et utilisés uniquement lors de l'initialisation de l'infrastructure. Les **rôles métier** sont créés spécifiquement pour le projet et régissent les accès quotidiens aux données. Cette architecture applique le principe du moindre privilège : chaque rôle dispose uniquement des droits strictement nécessaires à sa fonction.

3.1 Rôles système Snowflake

Rôle système	Utilisation dans le projet
SYSADMIN	Création du warehouse, de la database et des schémas ; attribution des privilèges
SECURITYADMIN	Création des rôles métier et des comptes utilisateurs
ACCOUNTADMIN	Configuration du Time Travel

3.2 Rôles pour les équipes analytiques

Rôle	Utilisateur (USER_)	Accès	Cas d'usage
TRANSFORMER	DEV	Tous schémas — lecture et écriture	Pipeline ETL dbt et ingestion Python
BI_ANALYST	BI_ANALYST	Lecture sur tables et vues SCHEMA_FINAL	Consultation des rapports BI
DATA_SCIENTIST	DATA_SCIENTIST	Lecture sur tables SCHEMA_STAGING et SCHEMA_FINAL	Analyse exploratoire des données
MART_CONSUMER	MART_CONSUMER	Lecture sur vues SCHEMA_FINAL	Consommation des agrégats analytiques

4. Procédure de test

4.1 Qualité des données

Tests dbt

Les tests dbt couvrent l'intégrité référentielle, l'unicité, la non-nullité, les plages de valeurs et la cohérence temporelle. Ils sont définis dans les fichiers YAML de schéma de chaque modèle — le détail par colonne est consultable dans la documentation dbt.

Exécutés automatiquement dans le pipeline principal après chaque ingestion mensuelle, à l'étape `dbt test`.

```
Finished running 220 data tests in 0 hours 2 minutes and 15.10 seconds

Completed successfully

Done. PASS=220 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=220
```

Figure 8: Résultat dbt test : 220 tests passés, 0 erreur

4.2 Qualité du code

Tous les outils ci-dessous s'exécutent automatiquement sur chaque push/PR vers dev et main via GitHub Actions.

Tests Python

La suite de tests Pytest comprend un fichier par module d'ingestion — scraping, upload, chargement, initialisation et fonctions utilitaires — garantissant une couverture complète du pipeline.

Les tests interagissant avec Snowflake utilisent des **mocks** via `unittest.mock` pour simuler la connexion et le curseur sans connexion réelle à la base de données. Cette approche garantit des tests rapides, reproductibles et indépendants de l'infrastructure. Chaque test suit le pattern **Arrange-Act-Assert** : préparation des mocks en amont, exécution de la fonction testée, puis vérification des résultats.

Les fonctions embarquent également des **doctests** consultables dans la documentation des fonctions Python.

SQLFluff : Linting automatique du code SQL avec le dialect Snowflake. Peut également être exécuté manuellement en local avant un push.

CodeQL : Analyse statique de sécurité du code Python

Dependabot : Mise à jour automatique des dépendances, trimestriellement

Semantic Release : Versioning automatique basé sur les conventional commits. Le numéro de version suit le format `MAJEUR.MINEUR.CORRECTIF` : `CORRECTIF` incrémente pour un correctif de bug, `MINEUR` pour une nouvelle fonctionnalité rétrocompatible, `MAJEUR` pour un changement incompatible. Le type est dérivé automatiquement du préfixe du commit — `fix:`, `feat:`, `feat!:`. À chaque merge sur main, la version est calculée, le tag Git créé et le changelog généré sans intervention manuelle. Cela garantit un historique lisible et un versionnement sans erreur humaine.

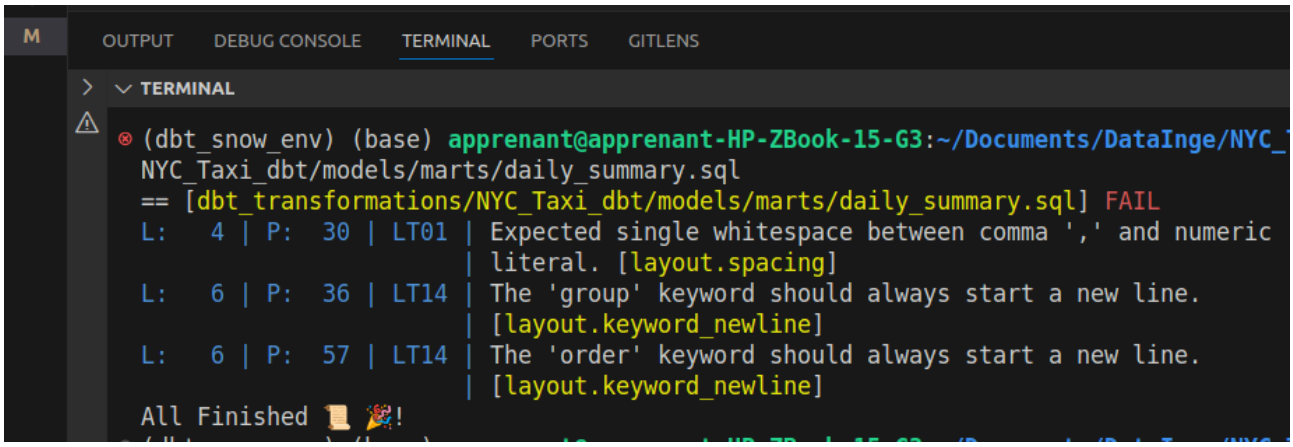

Figure 9: Exemple de linting SQLFluff sur un modèle dbt

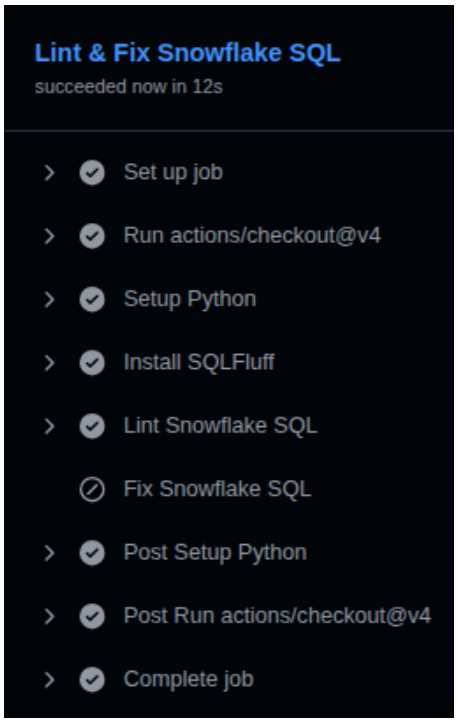

Figure 10: Workflow SQLFluff CI

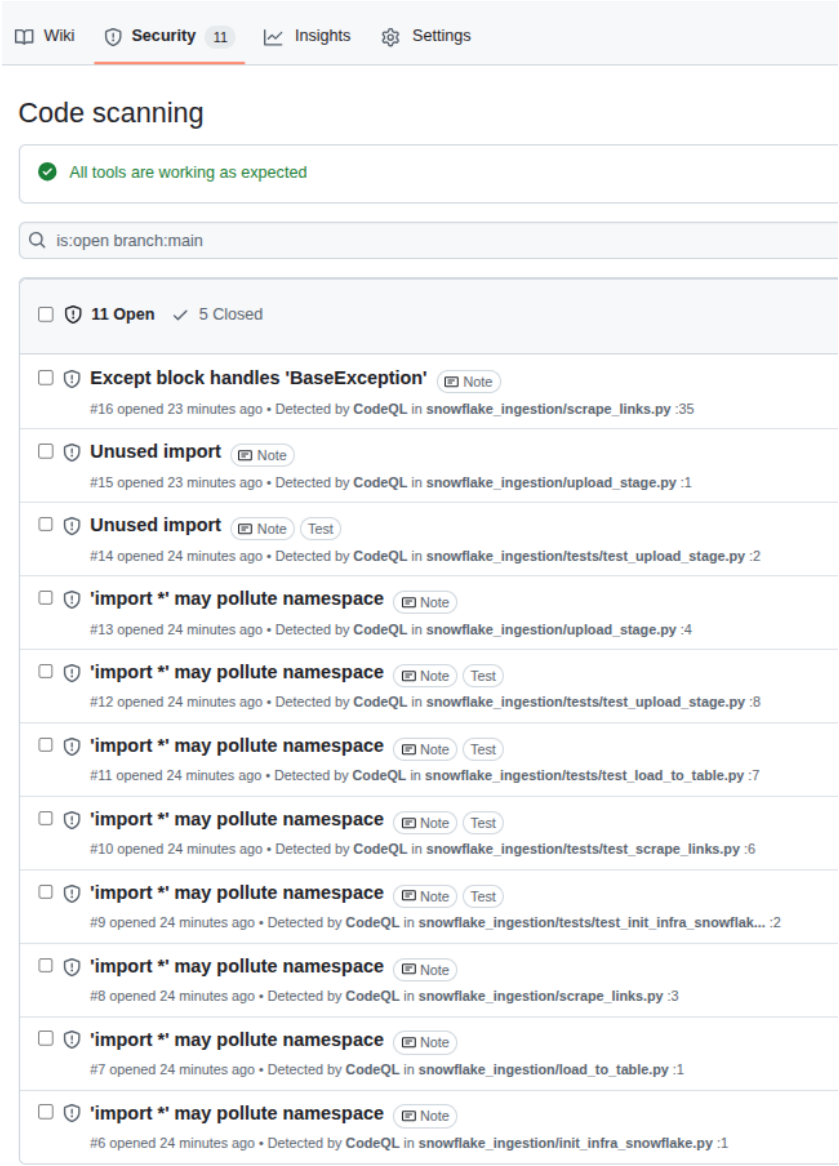

Figure 11: Analyse de sécurité CodeQL

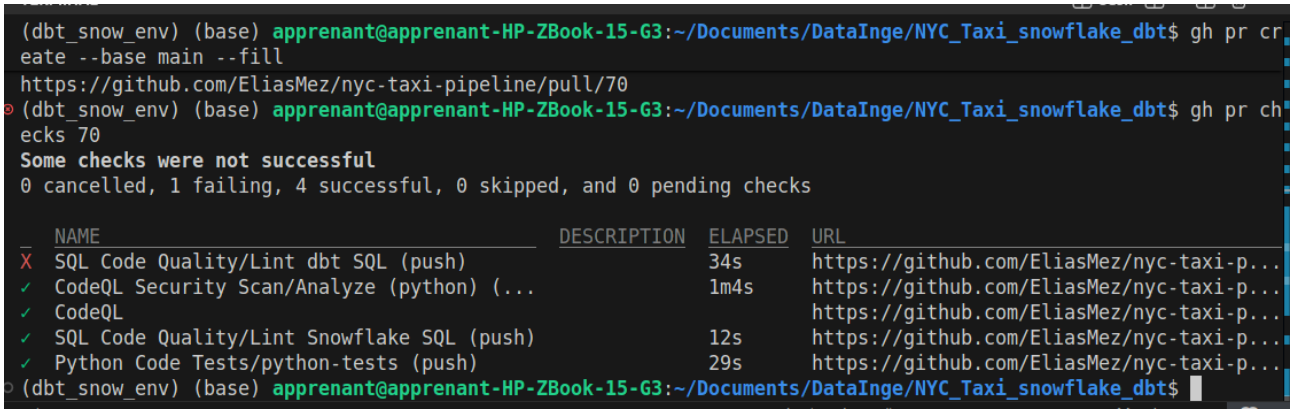

Figure 12: Checks CI sur une Pull Request

5. Documentation technique

5.1 Documentation MkDocs

La documentation du projet est déployée via **MkDocs Material** avec support multilingue en quatre langues, structurée par thématique pour une navigation rapide.

URL : https://eliasmez.github.io/nyc-taxi-pipeline/

La documentation des fonctions Python est générée automatiquement depuis les docstrings via **mkdocstrings**, couvrant les 6 modules d'ingestion et leurs fichiers de tests.

URL : https://eliasmez.github.io/nyc-taxi-pipeline/docstrings/

5.2 Documentation dbt

La documentation dbt est générée automatiquement depuis les fichiers YAML de schéma. Chaque modèle dispose d'une description et d'une description par colonne, avec les tests associés directement visibles dans l'interface. dbt docs génère également le graphe de dépendances complet, visualisant le flux depuis les sources jusqu'aux marts.

Couche	Modèles documentés
Sources	`yellow_taxi_trips_raw`
Staging	`yellow_taxi_trips_stg`, `int_trip_metrics`
Final	`fact_trips`, `dim_date`, `dim_time`
Marts	7 vues temporelles

URL : https://eliasmez.github.io/nyc-taxi-pipeline/dbt/

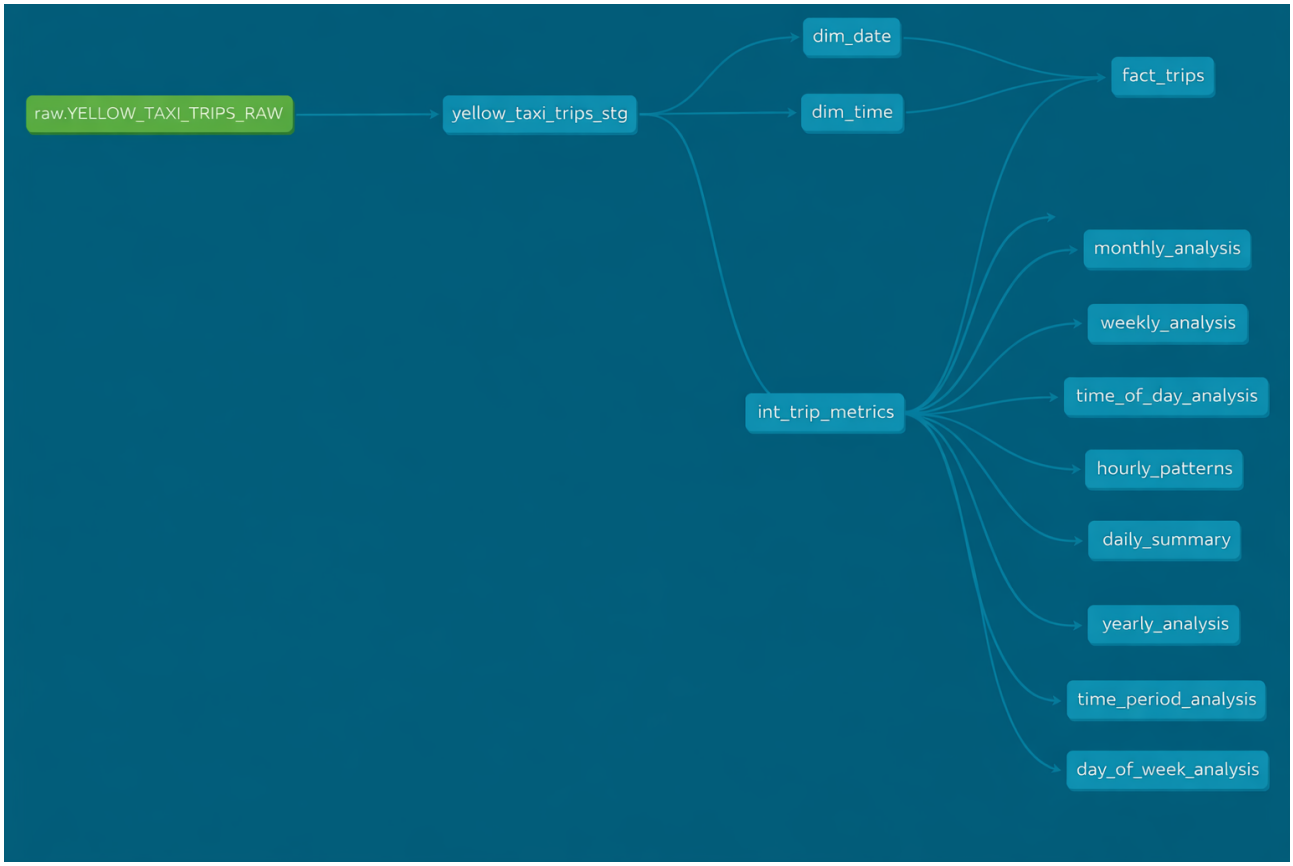

Figure 13: Lineage complet du projet dans dbt docs

5.3 Structure et accessibilité

Les deux interfaces sont consultables sur mobile et tablette.

MkDocs propose 3 palettes auto, clair et sombre avec détection de la préférence système, un support multilingue en 4 langues, une recherche avec suggestions et mise en surbrillance, et l'affichage de la date de dernière modification de chaque page. La structure par thématique permet d'accéder rapidement à une information spécifique.

dbt docs propose une recherche par modèle, colonne ou description, une navigation par couche et un graphe de dépendances interactif pour explorer le lineage.

6. Choix et retour d'expérience sur la pile technique

dbt a été retenu pour isoler la logique de transformation en SQL pur : cela facilite la lisibilité, la maintenance et la testabilité des modèles indépendamment du code d'orchestration. Ce principe d'isolation est appliqué de façon cohérente côté Python, où les requêtes SQL sont externalisées dans des fichiers .sql distincts du code applicatif. La gestion explicite des dépendances entre modèles a également motivé ce choix.

Snowflake a été choisi pour son scaling indépendant du stockage et du calcul, sa suspension automatique configurable, et son intégration native avec dbt.

GitHub Actions offre une exécution gratuite des workflows CI/CD. Le code étant hébergé sur GitHub, versionnement, orchestration et déploiement sont centralisés sur une seule plateforme. Les outils Dependabot, CodeQL et GitHub Pages étant nativement intégrés au dépôt, leur activation n'a requis aucune infrastructure tierce. Semantic Release s'intègre via une action standard de l'écosystème GitHub Actions.

MkDocs Material a été retenu pour la documentation : le plugin mkdocstrings permet de générer automatiquement la documentation des fonctions Python depuis les docstrings, le plugin i18n assure le support multilingue, et le déploiement sur GitHub Pages s'effectue sans infrastructure additionnelle.

Intégrer les ETL nécessaires en entrée et en sortie d'un entrepôt de données

1. Ingestion Python

Les scripts d'ingestion Python traitent des fichiers **Parquet** mensuels contenant plusieurs dizaines de millions de lignes.

`scrape_links.py` effectue une requête HTTP GET sur le site NYC TLC et extrait via XPath les liens des fichiers Parquet Yellow Taxi. `upload_stage.py` télécharge chaque fichier et l'uploade dans le stage Snowflake. Le fichier temporaire local est supprimé automatiquement après upload. `load_to_table.py` détecte dynamiquement le schéma du fichier Parquet dans le stage et crée la table RAW si elle n'existe pas encore. Le chargement s'effectue via `COPY INTO` vers `SCHEMA_RAW` avec injection de `TRIP_ID` depuis une séquence Snowflake. Le chargement est idempotent : une ré-exécution en cas d'échec ou de déclenchement manuel ne produit pas de doublons dans `SCHEMA_RAW`. Le fichier est ensuite supprimé du stage.

2. Transformations dbt

Les données transitent à travers trois zones — `SCHEMA_RAW`, `SCHEMA_STAGING`, `SCHEMA_FINAL` — et accumulent plusieurs centaines de millions de lignes dans la table de faits ; les dimensions et les marts restent compacts. Les transformations dbt appliquent les traitements nécessaires pour conformer les données aux schémas physiques définis dans le MPD. La conformité de `SCHEMA_RAW` est assurée en amont par la détection dynamique du schéma Parquet et l'injection de `TRIP_ID` lors du chargement. Le typage explicite de chaque colonne assure la conformité de `SCHEMA_STAGING`. Les clés surrogates et les jointures produisent `fact_trips`, `dim_date` et `dim_time` conformes au schéma en étoile de `SCHEMA_FINAL`.

2.1 Staging (`yellow_taxi_trips_stg.sql`)

Nettoyage : Chaque colonne est castée explicitement vers son type cible — timestamps convertis depuis leur format source, indicateurs booléens normalisés, montants avec précision décimale. Les lignes invalides sont écartées : distances négatives, montants nuls ou négatifs, timestamps absents ou incohérents. La déduplication est assurée par `trip_id` comme clé unique dbt : à chaque exécution incrémentale, les lignes déjà chargées sont ignorées.

2.2 Modèle dimensionnel

`fact_trips` extrait la date et l'heure depuis les timestamps de prise en charge et de dépose, puis les joint à `dim_date` et `dim_time` via des clés surrogates. La table est matérialisée de façon incrémentale avec `trip_id` comme clé unique.

`dim_date` consolide les dates distinctes de prise en charge et de dépose en une dimension plate, avec une clé surrogate au format YYYYMMDD et des attributs temporels couvrant l'année, le mois, le trimestre, le jour et le type de journée.

`dim_time` consolide les heures distinctes de la même façon, avec une clé surrogate au format HH24MISS et des attributs heure, minute, seconde.

2.3 Marts

7 vues pré-agrègent les données selon différents axes temporels : à la journée, à l'heure, au jour de la semaine, à la semaine, au mois, à l'année et par période de la journée. Chaque vue expose des métriques de volume, de distance et de revenu calculées depuis la vue intermédiaire `int_trip_metrics`. Toutes sont matérialisées dans `SCHEMA_FINAL`.

Synthèse

La modélisation du projet NYC Taxi repose sur un **schéma en étoile** suivant l'approche **bottom-up de Kimball**, adaptée au volume important de données, à la fréquence mensuelle de mise à jour et à la source unique NYC TLC. La table de faits `fact_trips`, reliée à deux dimensions — `dim_date` et `dim_time` — alimente **7 vues analytiques temporelles**. L'approche bottom-up est justifiée par le besoin de livrer rapidement de la valeur analytique sans modélisation préalable d'un modèle d'entreprise complet.

L'entrepôt de données **NYC_TAXI_DW** est déployé sur **Snowflake**, orchestré par **GitHub Actions** et transformé par **dbt** en 3 schémas : RAW, STAGING et FINAL. Quatre rôles avec des niveaux d'accès différenciés garantissent la sécurité des données. La procédure de test couvre l'intégrité référentielle, la qualité des données, les tests unitaires Python, le linting SQL et l'analyse de sécurité. La documentation est multilingue, structurée par thématique et déployée automatiquement. La pile technique est optimisée pour le volume et la fréquence mensuelle : auto-suspend du warehouse, tables STAGING transitoires, matérialisation incrémentale.

Le pipeline ETL comporte **6 étapes** — scraping, upload et chargement en Python ; staging, modèle dimensionnel et marts en dbt. Les données entrent au format **Parquet** depuis le site NYC TLC, transitent à travers 3 schémas et sont nettoyées par filtres sur les distances, montants et timestamps. Le typage est explicite et la déduplication assurée par clés uniques dbt. Les **7 marts temporels** fournissent des agrégats pré-calculés directement exploitables par les outils BI.